TECHNICAL DESIGN DOCUMENT
Township Operations Platform — Phase 8 Complete Release

Pune Gated Township (2,400 Apartments · 28 Towers · 180 Facility Staff)

Target System:	Pune Gated Township (2,400 Flats, 28 Towers, 180 Staff)
Architecture Model:	Option B: Supabase PostgreSQL 16 (RLS) + AWS Serverless Lambda
Security & Roles:	Strict 2-Role RBAC (RESIDENT, HELPDESK) + Granular Permissions + Force RLS
Financial Ledger:	64-bit Integer Paise + Append-Only Double-Entry Engine + Trigger Immutability
Gate & Communications:	Offline IndexedDB Tablet Sync + 90-Day S3 Photo Retention + Targeted Circulars
Amenity Reservations:	Asia/Kolkata Timezone + Row-Locking Concurrency Safety + 15m Expiration Lock

1. System Overview & Problem Statement

The platform coordinates township management across 2,400 apartments in Pune, Maharashtra. It features automated maintenance billing, multi-channel payment reconciliation (UPI/NEFT/Cheque), resident registry management, SLA-driven complaint ticketing, committee executive reporting, offline-first gate security tablet synchronization, multi-channel notification dispatch, official township notices, and race-condition-free amenity slot reservations. All financial operations utilize strict integer paise arithmetic.

2. Identity, RBAC & Multi-Tenant Security

The platform maintains strictly two application roles: RESIDENT and HELPDESK. Fine-grained permissions (COMPLAINT_MANAGE, BANK_IMPORT, RECONCILIATION_REVIEW, BILLING_MANAGE, COMMITTEE_REPORT_READ, GATE_OPERATIONS) govern operational capabilities. Multi-tenant isolation is enforced at the database engine level via PostgreSQL FORCE ROW LEVEL SECURITY across all application tables.

3. Core Subsystem Architectural Specifications

3.1 Resident Registry & Flat Allotments

Tracks Owner, Tenant, and Family Member memberships. Tenant leases enforce date validation (valid_until >= valid_from). Expired tenant leases automatically forfeit flat data access. Mandatory E.164 phone normalization is enforced.

3.2 Billing Engine & Append-Only Financial Ledger

Invoices snapshot unit rates at generation. Balances are derived dynamically from an append-only double-entry financial ledger. The late-fee engine supports Flat Fee, Monthly Percentage, and Daily Simple Interest in basis points. Row-level locking (SELECT ... FOR UPDATE) and idempotency keys protect payment allocations.

3.3 UPI-Aware Automated Bank Reconciliation

Bank statement imports undergo concurrent SHA-256 deduplication. An intelligent narration normalizer extracts UPI VPAs and fuzzy matches apartment identifiers. Matches with >= 90% confidence are promoted. A maker-checker verification modal enforces separation of duties prior to financial ledger settlement.

3.4 Service Desk & Persistent Safety Alerts

Complaints are auto-assigned via round-robin workload leveling. Emergency tickets feature a persistent duty-manager queue with a 15-minute response SLA. Staff can correct miscategorized tickets and link duplicates while preserving audit history.

3.5 Gate Management, Offline Sync & Photo Retention

Decoupled approval and entry state machines. Salted guest passes (GP-XXXX). Offline-first tablet architecture using IndexedDB with replay protection. 90-day visitor photo retention enforced via S3 lifecycle and purge workers.

3.6 Multi-Channel Communications & Official Notices

Outbox-driven notification engine over Amazon SQS queues with DLQs. Granular targeting for official notices by society, tower, and flat. Supports draft/publish states, auto-expiry, pinning, read tracking, and emergency outbox broadcast.

3.7 Amenity Booking & Concurrency Control

Discrete time-slot booking engine with Asia/Kolkata timezone awareness, PostgreSQL row locking (SELECT ... FOR UPDATE) preventing capacity oversubscription, 15-minute payment expiration lock, cancellation policies, and staff maintenance blocking.

4. Deployment & Infrastructure Architecture (Option B)

AWS CDK v2 provisions AWS Serverless compute (FastAPI on AWS Lambda with Mangum, Amazon API Gateway, Amazon SQS + DLQs, KMS-encrypted S3, and CloudFront CDN) paired with Supabase PostgreSQL 16 (Multi-AZ, RLS, PgBouncer port 6543 pooler). CI/CD pipelines enforce Ruff, Mypy strict typing, Pytest unit/integration/concurrency tests, Vitest, Gitleaks, and CodeQL SAST.